

Lab 1: Introduction to ESP32 and Basic Digital Outputs

Overview

In this lab, students will learn to set up their development environment for the ESP32, write basic programs to manipulate digital outputs, and explore how programming can control hardware like LEDs. ESP32 are a family of microcontrollers that are widely used in industry, and the ESP32-S3 used in this course has built-in Wi-Fi and Bluetooth capabilities as well as a dual-core architecture, making it ideal for a wide range of projects.

Objectives

- Successfully install and configure the Arduino IDE to program the ESP32.
- Understand the pin configuration and capabilities of the ESP32.
- Apply basic programming concepts to control digital outputs.
- Develop a simple program that manipulates an LED in a blinking pattern.
- Troubleshoot and debug common issues in setting up and programming the ESP32.



Figure 1: ESP32 (left) and its pinout (right)
[Datasheet \(Website\)](#)

References:

- [Lab Quick Reference](#)
- [Arduino IDE Setup](#)
- [Arduino Macros](#)

Part I: Programming Basics and Experimenting with Digital Outputs

In this step, you will write a simple program to blink an LED using the onboard LED of the ESP32. This exercise introduces you to basic programming concepts and how to manipulate digital outputs on a microcontroller. By the end of this section, you will understand how to write, upload, and run a basic sketch on your ESP32. To start, ensure you have the Arduino IDE and appropriate Board Manager package downloaded and installed ([steps here](#)).

Materials:

- ESP32 Development Board
- Breadboard
- Two LEDs (any color)
- Two 220 Ohm resistors
- Jumper wires

Arduino Library Functions

In every new Arduino sketch, you'll notice two methods appear automatically: `setup()` and `loop()`:

```
1 void setup() {  
2     // put your setup code here, to run once:  
3  
4 }  
5  
6 void loop() {  
7     // put your main code here, to run repeatedly:  
8  
9 }  
10
```

- The `setup()` function is called once when the program starts, and is where you set up the initial state of the microcontroller. It is used to initialize variables, pin modes, start using libraries, and perform any other setup tasks that need to be done before the main program loop begins.
- The `loop()` function is the core of your program. It handles the repeated tasks that the microcontroller performs, such as reading sensor values, checking inputs, controlling outputs, and running other repetitive tasks.

Outside of `setup()` and `loop()`, In Lab 1 we will primarily use three Arduino library functions:

- `pinMode(pin, mode)`: this function is used to configure a specific pin on the microcontroller to behave either as an input or an output.
- `digitalWrite(pin, value)`: this function is used to set a digital pin to a HIGH or LOW voltage level (i.e. turn it on or off).
- `delay(milliseconds)`: this function pauses the program for a specified amount of time.

Task:

- To start, ensure:
 - The Arduino IDE is open.
 - A new blank sketch is up.
 - Your ESP32 is connected via usb to your computer.
 - And the ESP32 is selected under "Select Board".
- The goal of this step is to blink the external LED connected to ESP32.
 - *Note:* alternatively, you can do this task with an internal LED by using Adafruit NeoPixel libraries and `setPixelColor()`. This requires additional libraries so feel free to just use an external LED.
- In this step, you will choose and define a General Purpose Input/Output (GPIO) pin to control the LED, providing an introduction to building basic circuits on a breadboard.
- A 220 Ω resistor is placed between the GPIO pin and the anode (long) leg of the LED to limit current, protecting both the LED and the microcontroller.
 - **IMPORTANT:** it is good practice to make sure the circuit is powered off and the ESP32 is not connected to your computer (or power) while making connections to avoid shorts and damaging components.

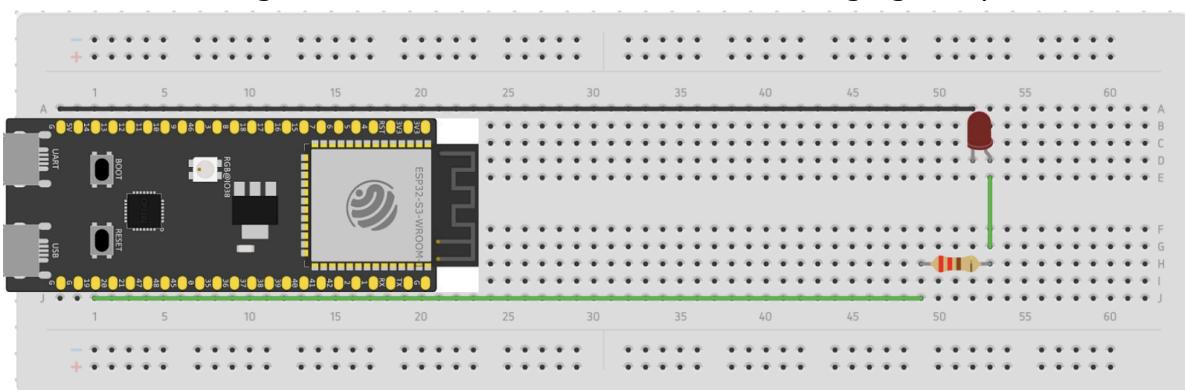
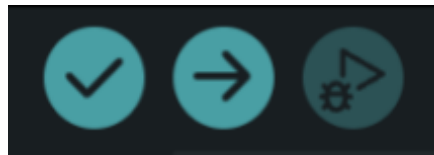


Figure 2: Controlling an external LED

- First we want to *initialize* the external LED pin (pin 19) to act as OUTPUT.
 - Then we want to tell the ESP32 to turn the LED on and then off repeatedly, with a one second delay between (i.e. 1 second on, 1 second off, 1 second on, ...).
- Once you've written your code, click the verify button (checkmark icon) in the Arduino IDE toolbar to ensure that the code compiles.
- To prepare the board for programming, you need to press and hold the 'BOOT' button, press and release the 'RESET' button, then release the 'BOOT' button on the board. Then press the run button (right arrow icon) on the Arduino IDE to upload the program to your ESP32 board.



- If the upload is successful, you will see a message like "Done uploading" in the status bar.
- Once the code is uploaded, the onboard LED on the ESP32 should start blinking on and off at one-second intervals.
- Try changing the delay values to different numbers (e.g. 500 for half a second or 2000 for two seconds) and upload the code again to see how it affects the blinking pattern.
- If successful, add a second LED connected to a different digital pin (with a 220 Ω resistor) and write your sketch to control both LEDs separately with each blinking at different frequencies.
 - Try to write your own functions to reduce repetitive code.
 - Connect the cathode side of both LEDs to common ground.

Deliverables for Part I:

- A live demo of the external (or built-in) LED blinking pattern on your ESP32.
- Your sketch code (with [comments](#))

Part II: Adding Inputs

In this step we will integrate both input and output to control the function of an LED through a tactile switch, allowing us to explore the functionality of the Arduino Nano ESP32's GPIO pins and laying the foundation for more complex embedded systems circuits in later labs.

Materials:

- ESP32 Development Board
- Breadboard
- LED (any color)
- 220 Ohm resistor
- Jumper wires
- Tactile switch

Arduino Library Functions

- In Part II we will introduce a new library function:
 - `digitalRead(pin)`: This function reads the value from a specified digital pin on the microcontroller. It returns HIGH (1) if the pin is at a high voltage level (typically 3.3V or 5V) and LOW (0) if the pin is at a low voltage level (0V).

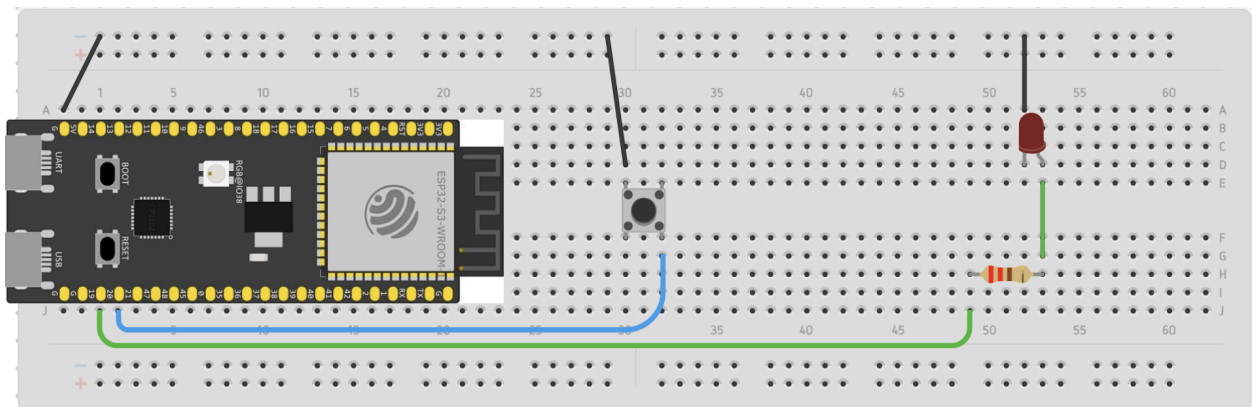


Figure 3: Controlling an external LED through a tactile switch

Task:

- Build the circuit according to *Figure 3* above.
- Write your sketch to control the external LED such that, when the button is pressed, the LED turns on, otherwise the LED remains off.
- Now connect your ESP32 to your computer, and click the run button to upload your code to the ESP32.
- *Hint:* refer to the [Lab Quick Reference](#) guide on Pullup Resistors for this section.

Debugging

- If you're having trouble, first refer to the [Debugging Tips](#) document. Debugging is a crucial skill in computer science and engineering, and the earlier you develop those skills the better.

Deliverables for Part II:

- A live demo of the tactile switch controlling the operation of the LEDs.
- Your sketch code (with [comments](#))

Part III: Extra Credit (up to 5% of total)

- Using everything we've covered in this lab, build a circuit and create a sketch to control multiple LEDs with tactile switches.
 - Set things up such that two or more LEDs are initially flashing.
 - Each LED will be controlled by its own switch. When that LED's switch is pressed (not held, just pressed), that LED turns off and remains off until the button is pressed again, at which point it begins flashing again.

Deliverables for Part III:

- A live demo of the buttons controlling the operation of the LEDs.
- Your sketch code (with [comments](#))

Conclusion

In this lab, you have gained hands-on experience with setting up the ESP32 development environment, writing and uploading basic programs to control digital outputs, and constructing simple circuits on a breadboard. By successfully installing the Arduino IDE, configuring ESP32 support, and using GPIO pins, tactile switches, and pullup resistors to control LEDs, you have learned foundational skills essential for embedded systems development.

Final Deliverables Recap

Live Demos:

- *Part I:* External (or built-in) LED blinking pattern on the ESP32.
- *Part II:* Button-controlled LED operation.
- *Part III:* Multiple button-controlled LEDs (Optional).

Code:

NOTE: Please follow the [Code Guidelines](#) when documenting your code.

The sketch files for the following sections should be descriptively named (e.g. Lab1Part2.ino), zipped together in a single folder containing your last name(s) and the lab number (e.g. LastNameLab1.zip), and submitted through Canvas:

- *Part I:* External (or built-in) LED blinking pattern on the ESP32.
- *Part II:* Button-controlled LED operation.
- *Part III:* Multiple button-controlled LEDs (Optional).

Grading

Each required section of this lab with deliverables (Parts I & II) will consist of the Live Demonstration and the Code portions:

- The Demonstrations will be graded as Complete or Incomplete and requires a visual confirmation by an Instructor for a total of **30 points** for Lab 1.
- In total, the code portion will be graded according to the following rubric:

Criteria	Exceptional (20 points)	Good (16-19 points)	Average (13-15 points)	Poor (0-12 points)
Code	• Thoroughly documented according to guidelines • Well-organized and efficient • Compiles without errors • Performs expected behavior consistently	• Mostly documented with minor unexplained sections • Mostly well-organized • Compiles, potentially with minor modifications. • Largely performs expected behavior with some minor deviations	• Some documentation is present, but significant sections are unexplained • Somewhat organized • Compiles with modifications • Some expected behavior, but inconsistencies present	• No documentation • Poor organization • Does not compile • Unclear what the code was attempting to do.

Total: 50 points